

C Programming 소개

Junho Kim

Kookmin University

C 프로그래밍 소개

C vs Python: 핵심 차이점

C vs Python: 핵심 차이점

Python과 C의 가장 큰 차이는 타입 선언, 컴파일, 메모리 관리입니다.

Python

```
1  # 타입 선언 불필요 (동적)
2  x = 10
3  name = "홍길동"
4
5  # 실행: python3 hello.py
6  print("Hello, World!")
7
8  # 메모리 자동 관리 (GC)
9  my_list = [1, 2, 3]
10
11 # 문자열 객체
12 s = "hello"
13 print(len(s))  # 5
```

C

```
1  // 타입 명시 필수 (정적)
2  int x = 10;
3  char name[] = "홍길동";
4
5  // 컴파일 후 실행
6  // gcc hello.c -o hello && ./hello
7  printf("Hello, World!\n");
8
9  // 메모리 수동 관리
10 int *arr = malloc(3 * sizeof(int));
11 free(arr);
12
13 // 문자열 = char 배열
14 char s[] = "hello";
15 printf("%zu\n", strlen(s));  // 5
```

C vs Python: 비교표

항목	Python	C
타입 선언	불필요 (동적)	필수 (정적)
실행 방식	인터프리터	컴파일 필요
메모리 관리	자동 (GC)	수동 malloc/free
문자열	<code>str</code> 객체	<code>char</code> 배열 + <code>\0</code>
표준 출력	<code>print()</code>	<code>printf()</code>
클래스/OOP	<code>class</code>	<code>struct</code> (메서드 없음)
문자열 비교	<code>s1 == s2</code>	<code>strcmp(s1, s2)</code>
배열 경계 검사	자동	없음 ⚠️

(이렇게 귀찮음에도) C를 배우는 이유

-  시스템 수준 프로그래밍 이해
-  메모리 구조 직접 제어
-  C++의 기반 — 오늘 배우는 것이 C++의 토대
-  운영체제, 임베디드, 게임 엔진의 언어

C 프로그래밍 소개

컴파일 & 빌드 과정

첫 번째 C 프로그램

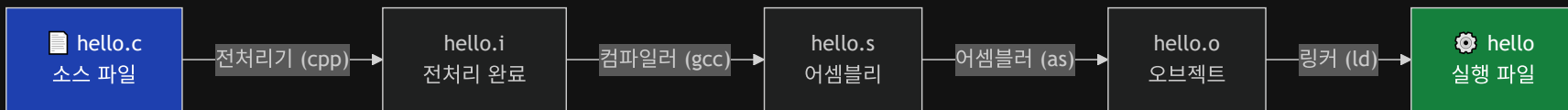
```
1  #include <stdio.h>    // 헤더 포함 (Python의 import)
2
3  int main(void) {      // 프로그램 진입점 (필수!)
4      printf("Hello, World!\n");
5      return 0;         // 0 = 정상 종료
6  }
```

컴파일 & 빌드 과정

```
1  # 기본 컴파일 & 링크 (한 번에)
2  gcc hello.c -o hello
3
4  # 권장 옵션 (경고 + C11 표준)
5  gcc -Wall -Wextra -std=c11 hello.c -o hello
6
7  # 실행
8  ./hello
```

자세한 컴파일 & 빌드 과정

소스 → 실행까지 4단계



주요 gcc 명령어

```
1  # 기본 컴파일 & 링크 (한 번에)
2  gcc hello.c -o hello
3
4  # 단계별 실행
5  gcc -E hello.c -o hello.i  # 전처리
6  gcc -S hello.c -o hello.s  # 어셈블리
7  gcc -c hello.c -o hello.o  # 오브젝트
8
9  # 권장 옵션 (경고 + C11 표준)
10 gcc -Wall -Wextra -std=c11 hello.c -o hello
11
12 # 실행
13 ./hello
```

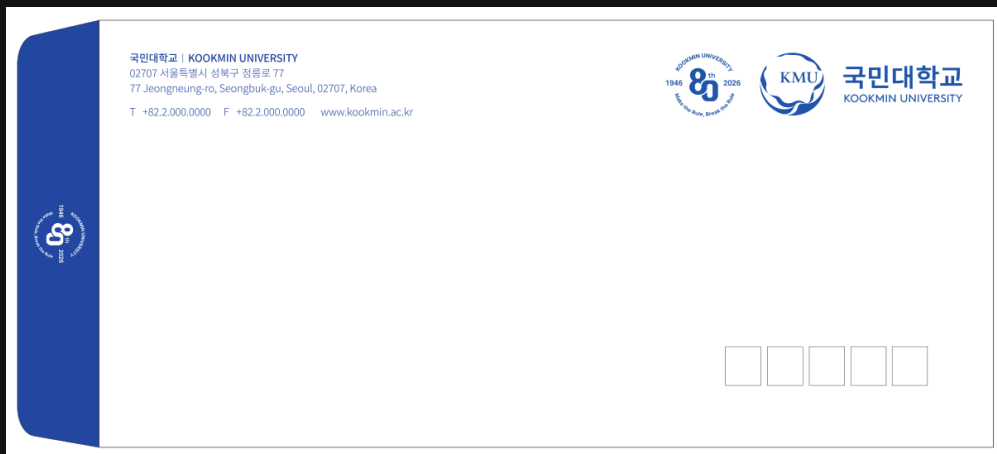

C 프로그래밍 소개

변수, 자료형, 연산자

Quiz

대한민국은 우편번호 기입을 위해 5칸을 사용합니다. 한 칸마다 0-9까지 십진수(decimal) 기입이 가능합니다.

😞 대한민국에서 5자리 우편번호로 만들 수 있는 서로 다른 우편번호는 몇가지인가요?



Quiz

컴퓨터에서 정보를 저장하는 기본단위는 byte입니다. 1 byte는 8칸으로 구성되어 있으며, 한 칸마다 0 또는 1 이진수(binary) 기입이 가능합니다.

😞 1 byte에 표현할 수 있는 서로 다른 수는 몇가지인가요?

0	1	1	0	0	0	0	0
---	---	---	---	---	---	---	---

😞 2 bytes에 표현할 수 있는 서로 다른 수는 몇가지인가요?

0	0	0	1	0	1	0	0
---	---	---	---	---	---	---	---

0	1	1	0	0	0	0	0
---	---	---	---	---	---	---	---

😞 4 bytes에 표현할 수 있는 서로 다른 수는 몇가지인가요?

0	1	0	0	1	0	0	1
---	---	---	---	---	---	---	---

1	0	1	0	1	0	1	0
---	---	---	---	---	---	---	---

0	0	0	1	0	1	0	0
---	---	---	---	---	---	---	---

0	1	1	0	0	0	0	0
---	---	---	---	---	---	---	---

기본 자료형

정수형 & 문자형

```
1 // 정수형
2 char c = 'A'; // 1 byte (-128 ~ 127)
3 short s = 100; // 2 bytes
4 int i = 42; // 4 bytes ← 가장 많이 씀
5 long l = 1000000L; // 8 bytes
6
7 // 부호 없는 정수
8 unsigned int u = 4294967295U;
9
10 // 실수형
11 float f = 3.14f; // 4 bytes (단정밀도)
12 double d = 3.14159; // 8 bytes (배정밀도) ← 권장
13
14 // 논리값 (C99)
15 #include <stdbool.h>
16 bool flag = true; // 내부적으로 1/0
```

크기 확인 & printf 형식 지정자

```
1 printf("char : %zu bytes\n", sizeof(char));
2 printf("int : %zu bytes\n", sizeof(int));
3 printf("double : %zu bytes\n", sizeof(double));
```

형식 지정자	자료형	예시
%d	int	printf("%d", 42)
%f	double	printf("%.2f", 3.14)
%c	char	printf("%c", 'A')
%s	char	printf("%s", "hi")
%p	포인터	printf("%p", ptr)
%zu	size_t	printf("%zu", sizeof(x))

연산자

산술 연산자

```
1  int a = 10, b = 3;
2
3  a + b    // 13
4  a - b    // 7
5  a * b    // 30
6  a / b    // 3 ← 정수 나눗셈!
7  a % b    // 1 ← 나머지
8
9  // Python처럼 실수 나눗셈 하려면
10 (double)a / b // 3.333...
```

비교 & 논리

```
1  // 비교 (결과는 1 또는 0)
2  a == b   // 0 (false)
3  a != b   // 1 (true)
4  a > b    // 1 (true)
5  a >= b   // 1 (true)
6
7  // 논리 (Python: and, or, not)
8  a > 0 && b > 0 // and
9  a > 0 || b > 0 // or
10 !(a > 0)      // not
```

증감 & 대입

```
1  int x = 5;
2
3  x++;    // x = 6 (후위 증가)
4  ++x;    // x = 7 (전위 증가)
5  x--;    // x = 6 (후위 감소)
6
7  // 복합 대입
8  x += 3;  // x = x + 3
9  x -= 1;  // x = x - 1
10 x *= 2;  // x = x * 2
11 x /= 4;  // x = x / 4
12 x %= 3;  // x = x % 3
```

⚠ Python과 다른 점: `int / int` 는 항상 정수! `10 / 3 = 3` (나머지 버림)

C 프로그래밍 소개

조건문 & 반복문

조건문

조건식의 논리 연산, 블록 처리 방식 등이 다름

C

```
1 // 기본 if-else
2 int score = 85;
3
4 if (score >= 90) {
5     printf("학점: A\n");
6 } else if (score >= 80) {
7     printf("학점: B\n"); // ← 출력됨
8 } else {
9     printf("학점: F\n");
10 }
```

- 조건식에 괄호 () 필수
- 블록에 중괄호 {} 권장 (한 줄도!)
- 각 문장 끝에 세미콜론 ; 필수

Python

```
1 # Python
2 if score >= 90:
3     print("학점: A")
4 elif score >= 80:
5     print("학점: B")
6 else:
7     print("학점: F")
```

- 조건식에 괄호 () 권장
- 블록은 들여쓰기(indentation) 필수
- 각 문장 끝에 세미콜론 ; 필요 없음

반복문

for 루프

```
1 // 기본 for
2 for (int i = 0; i < 5; i++) {
3     printf("%d ", i);
4 }
5 // 출력: 0 1 2 3 4
6
7 // 중첩 for: 구구단
8 for (int i = 2; i <= 9; i++) {
9     for (int j = 1; j <= 9; j++) {
10         printf("%d*%d=%d\n",
11             i, j, i*j);
12     }
13 }
```

while / do-while

```
1 // while
2 int n = 1;
3 while (n <= 5) {
4     printf("%d ", n);
5     n++;
6 }
7 // 출력: 1 2 3 4 5
8
9 // do-while (최소 1회 실행)
10 int x;
11 do {
12     printf("양수 입력: ");
13     scanf("%d", &x);
14 } while (x <= 0);
```

break & continue

```
1 // break: 루프 탈출
2 for (int i = 0; i < 10; i++) {
3     if (i == 5) break;
4     printf("%d ", i);
5 }
6 // 출력: 0 1 2 3 4
7
8 // continue: 다음 반복으로
9 for (int i = 0; i < 10; i++) {
10     if (i % 2 == 0) continue;
11     printf("%d ", i);
12 }
13 // 출력: 1 3 5 7 9
```

```
// Python의 range(5)와 비교: for (int i=0; i<5; i++) 가 동일
// Python의 enumerate는 C에서 인덱스 변수 i로 직접 관리
```


C 프로그래밍 소개

함수 정의와 호출

함수의 선언, 정의, 호출

C 프로그램

```
1 // ❶ 프로토타입 선언 (declaration)
2 int add(int a, int b);
3 int factorial(int n);
4
5 // main 함수 - 프로그램 진입점
6 int main(void) {
7     // ❷ 함수 호출 (call)
8     int r = add(3, 5); // add() 함수 호출
9     printf("r: %d\n", r);
10    printf("5! = %d\n", factorial(5)); // 120
11
12    return 0;
13 }
14
15 // ❸ 함수 정의 (definition)
16 int add(int a, int b) { return a + b; }
17
18 int factorial(int n) { // 재귀
19     if (n <= 1) return 1;
20     return n * factorial(n - 1);
21 }
```

Python 프로그램

```
1 # Python - 타입 선언 불필요
2 def add(a, b):
3     return a + b
4
5 def factorial(n):
6     if n <= 1: return 1
7     return n * factorial(n - 1)
```

C 함수의 특징

- 반환 타입 명시 필수 (`int` , `void` , `double` ...)
- 프로토타입 선언 필요 (사용 전에 알려줘야 함)
- 함수 오버로딩 없음 → C++에서 지원
- 값에 의한 전달, call(pass)-by-value — 원본 변경 불가

```
1 // 함수 프로토타입 선언 (declaration)
2 int fun(int, int);
3
4 // 함수 호출 (call)
5 int main(void) {
6     int a = 3; int b = 5;
7     int r = fun(a, b);
8     printf("a = %d, b = %d, r = %d\n", a, b, r);
9     return 0;
10 }
11
12 // 함수 정의 (definition)
13 int fun(int a, int b) {
14     a = 1; b = 2;
15     return a + b;
16 }
```

C 언어의 함수 인자 전달: 값 전달 (call-by-value)

💡 C는 모든 것이 기본적으로 값 전달(call-by-value)입니다.

참조 전달(call-by-reference)가 가능한 Python/Java/C++ 등과 달리, C는 값 전달만 가능하므로 원본을 바꾸려면 **포인터**를 사용해야 합니다.

❌ call-by-value (원본 안 바뀜)

```
1 void swap_wrong(int a, int b) {
2     int tmp = a;
3     a = b;
4     b = tmp;
5     // 지역 복사본만 바뀜!
6 }
7
8 int main(void) {
9     int x = 10, y = 20;
10    swap_wrong(x, y);
11    printf("%d %d\n", x, y); // 10 20 - 그대로!
12 }
```

✅ 포인터 기반 call-by-value (원본 바뀜)

```
1 void swap(int *a, int *b) {
2     int tmp = *a;
3     *a = *b;
4     *b = tmp;
5     // 주소를 받아 원본 직접 수정
6 }
7
8 int main(void) {
9     int x = 10, y = 20;
10    swap(&x, &y); // 주소를 넘김
11    printf("%d %d\n", x, y); // 20 10 ✅
12 }
```

C 프로그래밍 소개

포인터 기초

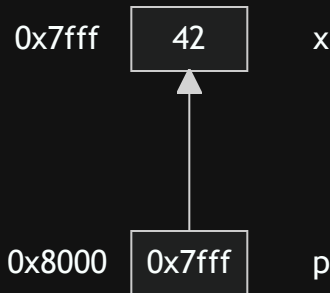
포인터란?

💡 포인터(pointer)는 말 그대로 가리키는 것!

포인터 기반 코딩

```
1 // 변수: 값을 저장하는 공간
2 int x = 42;
3
4 // 포인터: 메모리 주소를 저장하는 변수
5 int *p = &x; // & = 주소 연산자
6 // *p 타입: int를 가리키는 포인터
7 // p에 x의 주소가 저장됨
8
9 // 값 출력
10 printf("x = %d\n", x); // 42
11 printf("&x = %p\n", &x); // 0x7fff... (주소)
12 printf("p = %p\n", p); // 0x7fff... (동일!)
13 printf("*p = %d\n", *p); // 42 (역참조)
14
15 // 포인터로 값 변경
16 *p = 100; // *p = 역참조 연산자
17 printf("x = %d\n", x); // 100 ← x가 바뀜!
18 // p를 통해 x를 직접 수정한 것
```

메모리 시각화



*p = 100; // p가 가리키는 위치(0x7fff)에 저장된 값을 100으로 변경

포인터 사용의 철칙

- 이미 존재하는 것을 가리켜야 함
- 꼭 필요한 경우가 아니면 쓰지 말 것!!!
- 굳이 쓴다면 문법 구조를 이해하고 쓰자.

포인터 사용의 철칙

💡 포인터(pointer)는 말 그대로 가리키는 것!

- 이미 존재하는 것을 가리켜야 함
- 꼭 필요한 경우가 아니면 쓰지 말 것!!!
- 굳이 쓴다면 문법 구조를 이해하고 쓰자.

핵심 연산자 정리

연산자	이름	의미
<code>&x</code>	주소 연산자	x가 저장된 메모리 주소
<code>*p</code>	역참조 연산자	p가 가리키는 주소의 값
<code>int *p</code>	포인터 선언	p는 int를 가리키는 포인터

⚠ 포인터 관련 모든 문제는 포인터 연산의 직관적이지 못한 문법 구조 때문.

포인터 선언과 역참조 연산자의 표현이 동일... 이것이 모든 불행의 씨앗 😞

swap 구현 다시 보기: (call-by-value)

💡 C는 모든 것이 기본적으로 값 전달(call-by-value)입니다.

참조 전달(call-by-reference)가 가능한 Python/Java/C++ 등과 달리, C는 값 전달만 가능하므로 원본을 바꾸려면 **포인터**를 사용해야 합니다.

❌ call-by-value (원본 안 바뀜)

```
1 void swap_wrong(int a, int b) {
2     int tmp = a;
3     a = b;
4     b = tmp;
5     // 지역 복사본만 바뀜!
6 }
7
8 int main(void) {
9     int x = 10, y = 20;
10    swap_wrong(x, y);
11    printf("%d %d\n", x, y); // 10 20 - 그대로!
12 }
```

✅ 포인터 기반 call-by-value (원본 바뀜)

```
1 void swap(int* a, int* b) {
2     int tmp = *a;
3     *a = *b;
4     *b = tmp;
5     // 주소를 받아 원본 직접 수정
6 }
7
8 int main(void) {
9     int x = 10, y = 20;
10    swap(&x, &y); // 주소를 넘김
11    printf("%d %d\n", x, y); // 20 10 ✅
12 }
```


C 프로그래밍 소개

변수 Scope (유효 범위)

변수 Scope란?

변수가 선언된 위치에 따라 접근 가능한 범위가 결정됩니다.

```
1 // 전역 변수 (Global) - 파일 전체에서 접근 가능
2 int global = 100;
3
4 void funcA(void) {
5     // 지역 변수 (Local) - 이 함수 안에서만 유효
6     int local = 10;
7     global = 200; // ✅ 전역 변수 접근 가능
8     printf("%d\n", local); // 10
9     printf("%d\n", global); // 200
10 } // ← local은 여기서 소멸 (Stack에서 제거)
11
12 void funcB(void) {
13     // printf("%d\n", local); // ❌ 컴파일 오류!
14     printf("%d\n", global); // ✅ 200 (전역은 가능)
15 }
```

```
1 int main(void) {
2     int x = 42; // main의 지역 변수
3
4     { // 블록 scope
5         int y = 99;
6         printf("%d\n", x); // ✅ 42
7         printf("%d\n", y); // ✅ 99
8     }
9     // printf("%d\n", y); // ❌ y는 블록 밖에서 소멸!
10
11     funcA();
12     funcB();
13     return 0;
14 }
```

Scope 종류

Scope를 이해해야 포인터가 왜 필요한지 명확히 알 수 있습니다.

```
1 // 전역 변수 (Global) - 파일 전체에서 접근 가능
2 int global = 100;
3
4 void funcA(void) {
5     int local = 10;          // 지역 변수 (Local) - 이 함수 안에서만 유효
6     static int sss = 200;    // 정적 변수 (static) - 프로그램 종료까지
7
8     printf("%d\n", local);   // 10
9 } // ← local은 여기서 소멸 (Stack에서 제거)
10
11 int main(void) {
12     int x = 42;              // main의 지역 변수
13
14     { // 블록 scope
15         int y = 99;
16         printf("%d\n", y); // ✅ 99
17     }
18     return 0;
19 }
```

Scope 종류 한눈에 보기

종류	선언 위치	유효 범위	수명
전역	함수 밖	파일 전체	프로그램 종료까지
지역	함수 안	해당 함수	함수 반환 시 소멸
블록	{ } 안	해당 블록	블록 종료 시 소멸
정적	static	지역과 동일	프로그램 종료까지

💡 변수의 수명(life-time)과 유효 범위(scope)을 혼동하지 말자!

swap 구현 다시 보기: (call-by-value)

💡 C는 모든 것이 기본적으로 값 전달(call-by-value)입니다.

참조 전달(call-by-reference)가 가능한 Python/Java/C++ 등과 달리, C는 값 전달만 가능하므로 원본을 바꾸려면 **포인터**를 사용해야 합니다.

❌ call-by-value (원본 안 바뀜)

```
1 void swap_wrong(int a, int b) {
2     int tmp = a;
3     a = b;
4     b = tmp;
5     // 지역 복사본만 바뀜!
6 }
7
8 int main(void) {
9     int x = 10, y = 20;
10    swap_wrong(x, y);
11    printf("%d %d\n", x, y); // 10 20 - 그대로!
12 }
```

✅ 포인터 기반 call-by-value (원본 바뀜)

```
1 void swap(int *a, int *b) {
2     int tmp = *a;
3     *a = *b;
4     *b = tmp;
5     // 주소를 받아 원본 직접 수정
6 }
7
8 int main(void) {
9     int x = 10, y = 20;
10    swap(&x, &y); // 주소를 넘김
11    printf("%d %d\n", x, y); // 20 10 ✅
12 }
```

포인터 사용의 철칙

코딩 규칙: 포인터 선언과 역참조 연산자의 표현을 구분해서 쓰자!!!

```
1  int* p; // 포인터 변수 선언
2  int a = 10;
3  p = &a;
4  printf("%d", *p); // 역참조 연산
```

코딩 규칙 (구분된 규칙)

연산자	이름	의미
<code>&x</code>	주소 연산자	x가 저장된 메모리 주소
<code>*p</code>	역참조 연산자	p가 가리키는 주소의 값
😞 <code>int *p</code>	포인터 선언	p는 int를 가리키는 포인터
😊 <code>int* p</code>	포인터 선언	p는 int를 가리키는 포인터

✅ 포인터 기반 call-by-value (원본 바뀜)

```
1  void swap(int* a, int* b) {
2      int tmp = *a;
3      *a = *b;
4      *b = tmp;
5      // 주소를 받아 원본 직접 수정
6  }
7
8  int main(void) {
9      int x = 10, y = 20;
10     swap(&x, &y); // 주소를 넘김
11     printf("%d %d\n", x, y); // 20 10 ✅
12 }
```

C 프로그래밍 소개

배열과 문자열

배열 (Array)

```
1 // 배열 선언과 초기화
2 int arr[5] = {10, 20, 30, 40, 50};
3 // 인덱스:   0   1   2   3   4
4 printf("%d\n", arr[2]); // 30
5
6 // 배열 순회
7 int n = sizeof(arr) / sizeof(arr[0]); // 길이: 5
8 for (int i = 0; i < n; i++) {
9     printf("%d ", arr[i]);
10 }
11 // 출력: 10 20 30 40 50
12
13 // 2차원 배열 (개인적인 의견: 되도록 쓰지 마세요)
14 int matrix[2][3] = {
15     {1, 2, 3},
16     {4, 5, 6}
17 };
18 printf("%d\n", matrix[1][2]); // 6
19
20 // 배열 = 함수에 전달 시 포인터로 넘어감
21 void print_arr(int arr[], int n);
```

⚠ 배열 주의사항

1. 크기 고정 — 컴파일 시 결정, 변경 불가

```
int arr[5]; // 크기 5, 이후 변경 불가
```

2. 경계 검사 없음 — 가장 위험한 C 버그!

```
int arr[5];
arr[10] = 99; // ❌ 컴파일은 됨! 런타임 오류
// Python은 IndexError로 잡아주지만 C는 그냥 실행됨
```

3. 길이 계산 관용구

```
int n = sizeof(arr) / sizeof(arr[0]);
```

4. 함수에 배열 전달 시 크기도 같이 전달

```
void func(int arr[], int n) { ... }
func(arr, 5);
```

문자열 (String)

C 언어에서 문자열은 character 배열로 처리됨.

```
1 // 문자열 = char 배열 + NULL 종료자('\0')
2 char s1[10] = "Hello";
3 // 메모리: H e l l o \0 ? ? ?
4 // 인덱스: 0 1 2 3 4 5
5
6 // 문자열 리터럴 (수정 불가)
7 const char *s2 = "World"; // 읽기 전용!
8
9 // string.h 함수들
10 #include <string.h>
11
12 strlen(s1); // 길이: 5 ('\0' 미포함)
13 strcpy(dst, src); // 복사 (dst 충분히 커야 함)
14 strncpy(dst, src, n); // 안전한 복사
15 strcat(s1, "!"); // 연결: "Hello!"
16 strcmp(s1, s2); // 비교: <0, 0, >0
17
18 // 문자열 → 숫자 (#include <stdlib.h>)
19 int n = atoi("42"); // "42" → 42
20 double d = atof("3.14"); // "3.14" → 3.14
21
22 // 형식 지정 문자열 생성
23 char buf[50];
24 sprintf(buf, "%s is %d", "age", 20);
```

메모리 구조

```
char s1[10] = "Hello";
```

```
s1:    [0][1][2][3][4][5][6][7]
value: H e l l o \0 ? ?
                ↑
            NULL 종료자
            (필수! 없으면 오작동)
```

```
const char *s2 = "World";
```

```
s2:    [0][1][2][3][4][5][6][7]
value: W o r l d \0 ? ?
                ↑
            NULL 종료자
            (필수! 없으면 오작동)
```


포인터 & 배열

```
1  int arr[] = {10, 20, 30, 40, 50};
2
3  // 배열 이름 = 첫 번째 원소의 주소
4  int *p = arr;      // == &arr[0]
5  printf("%d\n", *p); // 10
6
7  // 포인터 산술
8  p++;               // 다음 int로 이동 (+4 bytes)
9  printf("%d\n", *p); // 20
10
11 // 두 표현은 동일
12 printf("%d\n", arr[2]); // 30
13 printf("%d\n", *(arr+2)); // 30
14
15 // 포인터로 배열 순회
16 for (int* ptr = arr; ptr < arr + 5; ptr++) {
17     printf("%d ", *ptr);
18 }
19 // 출력: 10 20 30 40 50
```

포인터 산술 규칙

```
1  arr = &arr[0] = 0x1000
2
3  p = arr      → p = 0x1000 (arr[0] = 10)
4  p + 1        → p = 0x1004 (arr[1] = 20) +4 bytes
5  p + 2        → p = 0x1008 (arr[2] = 30) +8 bytes
```

NULL 포인터

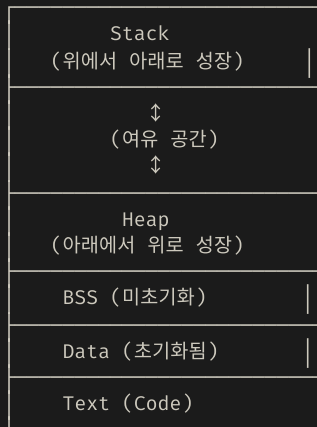
```
1  int* p = NULL; // 아무것도 가리키지 않음
2
3  // 사용 전 반드시 NULL 체크!
4  if (p != NULL) {
5      printf("%d\n", *p);
6  }
7
8  // NULL 포인터 역참조 → 프로그램 충돌!
9  // int* p = NULL;
10 // *p = 42; // ❌ Segmentation Fault
```

C 프로그래밍 소개

메모리 구조 이해

프로그램의 메모리 구조

1 높은 주소



← 지역 변수, 함수 호출
자동 할당 / 자동 해제

← 동적 메모리
malloc() / free()

← 초기화 안 된 전역변수

← 초기화된 전역/정적 변수

← 실행 코드 (읽기 전용)

18 낮은 주소

```
1 // Text 영역: 함수 코드
2 int add(int a, int b) { return a + b; }
3
4 // Data 영역: 초기화된 전역변수
5 int global_init = 100;
6 // BSS 영역: 초기화 안 된 전역변수
7 int global_uninit;
8
9 // Stack 영역: 지역변수 (함수 호출 시)
10 void func(void) {
11     int local = 42; // Stack에 할당
12     int arr[100]; // Stack에 할당 (400 bytes)
13     // 함수 반환 시 자동 해제
14 }
15
16 // Heap 영역: 동적 할당
17 #include <stdlib.h>
18
19 int* p = (int*)malloc(sizeof(int));
20 *p = 42;
21 free(p); // ← 반드시 해제!
22 p = NULL;
```

동적 메모리 할당

- 동적 메모리 할당은 바이트(byte) 단위로 할당됨.
- 할당된 메모리는 타입이 없음
- 포인터 타입이 할당된 메모리를 어떻게 쓰는지 결정함.
- 프로그램이 구동된 후에야 메모리의 크기가 결정될 때만, 동적 메모리 할당을 쓰도록 하자!

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(void)
5  {
6      int arr[100]; // 100개 정도는 스택에 잡으세요.
7      int* s = (int*)malloc(sizeof(int)*100); // 이런 코드는 쓰지 말 것!!!
8
9      void* p = malloc(8000);
10
11     char* pc = (char*) p;
12     int* pi = (int*) p;
13     float* pf = (float*) p;
14     double* pd = (double*) p;
15
16     return 0;
17 }
```

동적 메모리 할당 & 메모리 오류

올바른 동적 메모리 사용

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(void) {
5      int n = 5;
6      int* arr = (int*)malloc(n * sizeof(int));
7
8      // ✅ NULL 체크 필수
9      if (arr == NULL) {
10         fprintf(stderr, "메모리 할당 실패\n");
11         return 1;
12     }
13
14     for (int i = 0; i < n; i++)
15         arr[i] = (i + 1) * 10;
16
17     for (int i = 0; i < n; i++)
18         printf("%d ", arr[i]); // 10 20 30 40 50
19
20     free(arr); // ✅ 반드시 해제
21     arr = NULL; // ✅ 댕글링 포인터 방지
22     return 0;
23 }
```

흔한 메모리 오류

1. 메모리 누수 (Memory Leak)

```
1  int* p = malloc(100);
2  // free(p) 빠뜨림 → 프로그램이 계속 메모리 점유
```

2. 댕글링 포인터 (Dangling Pointer)

```
1  int* p = malloc(sizeof(int));
2  free(p);
3  *p = 42; // ❌ 해제된 메모리 접근! 위험!
```

3. 버퍼 오버플로우

```
1  char buf[5];
2  strcpy(buf, "Hello, World!"); // ❌ 넘침!
3  // 보안 취약점의 주요 원인
```

4. 이중 해제 (Double Free)

```
1  free(p);
2  free(p); // ❌ 이미 해제된 메모리 재해제
```

C 프로그래밍 소개

구조체 (struct)

구조체 정의 & 사용

구조체는 사용자 정의 자료구조임.

```
1 // 구조체 정의 (typedef로 편하게)
2 typedef struct {
3     char    name[50];
4     int     student_id;
5     double  gpa;
6 } Student; // ← 타입 이름
7
8 // 구조체 변수 생성 & 초기화
9 Student s1 = {"김철수", 20230001, 3.8};
10 Student s2; // 선언만
11
12 // 멤버 접근: . (점 연산자)
13 strcpy(s2.name, "이영희");
14 s2.student_id = 20230002;
15 s2.gpa = 3.5;
16 printf("%s: %.1f\n", s1.name, s1.gpa);
17
18 Student* s = &s1;
19 printf("%-10s %d %.2f\n",
20         s->name, // -> 연산자!
21         s->student_id,
22         s->gpa);
```

접근 연산자 비교

상황	연산자	예시
일반 변수	. (점)	s1.gpa
포인터 변수	-> (화살표)	p->gpa

- C struct 는 메서드(함수) 없음
 - C struct 는 상속 없음
- C++의 class = struct + 메서드 + 접근 제어

구조체 — 동적 할당 & 실전 예제

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  typedef struct {
6      char    name[50];
7      int     scores[3]; // 중간, 기말, 과제
8      double  average;
9  } Student;
10
11 // 평균 계산 함수
12 void calc_avg(Student* s) {
13     int sum = 0;
14     for (int i = 0; i < 3; i++)
15         sum += s->scores[i];
16     s->average = (double)sum / 3;
17 }
18
19 // 학점 반환
20 char get_grade(double avg) {
21     if (avg >= 90) return 'A';
22     if (avg >= 80) return 'B';
23     if (avg >= 70) return 'C';
24     return 'F';
25 }
```

```
1  int main(void) {
2      // 동적 배열로 학생 관리
3      int n = 3;
4      Student* students =
5          (Student*)malloc(n * sizeof(Student));
6
7      // 데이터 설정
8      strcpy(students[0].name, "김철수");
9      students[0].scores[0] = 85;
10     students[0].scores[1] = 90;
11     students[0].scores[2] = 78;
12
13     for (int i = 0; i < n; i++) {
14         calc_avg(&students[i]);
15         printf("%-10s 평균: %.1f 학점: %c\n",
16             students[i].name,
17             students[i].average,
18             get_grade(students[i].average));
19     }
20
21     free(students); // 반드시 해제!
22     return 0;
23 }
```

💡 이 패턴이 C++의 `class` + 생성자/소멸자로 발전합니다.

C 학습 완료 체크리스트

- ✓ 컴파일 & 빌드 — gcc, 전처리 링크 이해
- ✓ 변수 & 자료형 — 타입 명시, sizeof
- ✓ 조건문 & 반복문 — 괄호, 세미콜론 필수
- ✓ 함수 — 프로토타입, 값 전달 vs 포인터 전달
- ✓ 배열 & 문자열 — char 배열, string.h 함수
- ✓ 포인터 — &, *, 포인터 산술
- ✓ 메모리 구조 — Stack/Heap, malloc/free
- ✓ 구조체 — typedef struct, . vs ->

이제 C++로! 

다음에 배울 내용:

- `class` & 객체 지향 프로그래밍
- 생성자 / 소멸자
- 상속과 다형성
- 참조자 (`int &ref = x`)
- 연산자 오버로딩
- STL — `vector`, `map`, `string`
- 스마트 포인터 (`unique_ptr`, `shared_ptr`)

오늘 배운 포인터 & 메모리 이해가
C++를 잘 쓰기 위한 핵심 기반입니다!